

## MACHINE LEARNING RESEARCH PROJECT

# GeoPulse

Forecasting New York City bike-share demand, and turning the forecast into something a person can actually use

---

|                 |                                                                                                            |
|-----------------|------------------------------------------------------------------------------------------------------------|
| Data            | 79,410,195 Citi Bike trips · January 2023 – December 2024                                                  |
| Models          | LightGBM, Temporal Fusion Transformer, Spatio-Temporal Graph Neural Network, Seasonal Naive                |
| Spatial systems | Uber H3 hexagons · Google S2 quadrilaterals                                                                |
| Deliverable     | Eight completed phases, a tested prediction interface, and a map-based assistant                           |
| Code            | 11,238 lines across <code>src/</code> and <code>scripts/</code> · 2,008 lines of tests · 166 tests passing |
| Repository      | 12 commits on <code>main</code> , one per completed phase or feature                                       |

# Contents

---

## Overview

1. What this project is, in plain words
2. The four questions we set out to answer
3. How the whole system fits together

## Building the dataset

4. Where the data came from
5. Cleaning it without hiding anything
6. The timezone problem nobody expects
7. Turning trips into a grid of numbers

## Teaching a model to forecast

8. The baseline that had to be beaten
9. Building features, and proving they don't cheat
10. Which features actually earned their place
11. Choosing the size of a map cell
12. Hexagons versus squares

## The models

13. LightGBM, tuned properly
14. The two deep-learning models

15. Training on a free GPU
16. The final exam: the test set
17. Where the model gets things wrong

## From forecast to decision

18. Estimating how many docks exist
19. Moving bikes around, and why it didn't help
20. The prediction interface

## The application

21. Architecture of the app
22. From a cell forecast to a station answer
23. Finding places without a maps API
24. The assistant and its tools
25. Walking directions with A\*
26. The interface

## Reflection

27. Bugs worth remembering
28. Honest limitations
29. How to run everything
30. Glossary

## 1. What this project is, in plain words

---

New York City runs a bike-share system called Citi Bike. There are roughly 2,400 docking stations scattered across Manhattan, Brooklyn, Queens and Jersey City. People take a bike from one dock and return it to another. Millions of times a year.

This creates a problem that sounds simple and isn't. Bikes pile up where people ride *to*, and disappear from where people ride *from*. Every weekday morning, thousands of bikes flow from residential areas into Midtown. By 9:30am the residential docks are empty and the Midtown docks are full. Somebody has to drive a truck around and move them back.

**GeoPulse tries to predict that flow before it happens.** Given a place and a time, it forecasts how many bikes will be taken and how many will be returned over the next fifteen minutes to one hour. If you can predict it, you can act on it — either as an operator deciding where to send a truck, or as a rider deciding which dock to walk to.

The project has two halves that are quite different in character:

1

**A research project**

Eight phases of careful work: acquire the data, clean it, build features, train four different kinds of model, and measure honestly which one wins and why. This half produces numbers and conclusions.

2

**A working application**

A map-based assistant where you type "will I find a bike near Times Square?" and get a real answer from the trained models, with walking directions and an explanation. This half makes the research usable.

**THE RULE THAT SHAPED EVERYTHING**

One principle runs through the entire project: **never claim more than the data supports**. Several times this made the result less impressive and the report longer. Every one of those moments is written up here rather than quietly smoothed over, because a forecast you can't trust is worse than no forecast at all.

## 2. The four questions we set out to answer

The project was designed around four research questions. Each one was decided by an experiment, not by intuition.

| #          | Question                                        | Answer                                                                                                                                                                                                                                                            |
|------------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RQ1</b> | How big should a map cell be?                   | <b>H3 resolution 8</b> — cells of about 0.74 km <sup>2</sup> . Chosen using a <i>skill score</i> , because every ordinary accuracy metric is biased by cell size and the three we tried disagreed with each other completely.                                     |
| <b>RQ2</b> | Hexagons (H3) or squares (S2)?                  | <b>Almost a coin flip</b> . S2 level 13 scores slightly better on skill, but its cells are 47% larger, which plausibly explains the whole gap. The size of the cell matters; the shape barely does.                                                               |
| <b>RQ3</b> | Do deep-learning models beat gradient boosting? | <b>No</b> . A tuned LightGBM beat both the Transformer and the Graph Neural Network at both cell sizes. Two apparent deep-learning wins at H3-8 reversed at S2-13, which means they were noise. An important caveat about training budget is given in section 16. |
| <b>RQ4</b> | Do better forecasts produce better operations?  | <b>Yes between forecasts, no in absolute terms</b> . LightGBM out-performed even a <i>perfect</i> forecast at rebalancing. But rebalancing itself didn't help at all, for a structural reason explained in section 19.                                            |

**The headline numbers****1.14**

BEST TEST MAE (BIKES  
PER 15 MIN)

**38%**

BETTER THAN THE NAIVE  
BASELINE

**0.72**

HOTSPOT F1 SCORE

**166**

AUTOMATED TESTS  
PASSING

MAE means "mean absolute error" — on average, the forecast for a cell is off by 1.14 bikes in a 15-minute window. Hotspot F1 measures how well the model identifies the busiest 10% of cells at each moment, which is what an operator actually cares about.

### 3. How the whole system fits together

---

Before the details, here is the shape of the whole thing. Data flows downward; each stage depends only on the ones above it.

- 1 Raw data → typed Parquet files**  
90 CSV files (15 GB) of trip records, plus weather, public events, and traffic sensors. Converted to a compressed columnar format that can be queried without loading it all into memory.
- 2 Cleaning → 79.2 million usable trips**  
Nine named rules remove impossible records. Every removal is counted and attributed, so the numbers reconcile exactly.
- 3 Spatial indexing → a grid of cells**  
Every trip start and end is assigned to a hexagonal cell. Cells with too little activity are dropped.
- 4 The panel → 104 million rows of counts**  
One row per cell per 15-minute interval, holding how many bikes were taken and returned. This is the table everything else is built from.
- 5 Features → 168 columns describing each moment**  
Recent demand, time of day, weather, events, traffic, neighbouring cells. Only features that could genuinely be known at forecast time.
- 6 Models → four families trained and compared**  
Seasonal Naive, LightGBM, Temporal Fusion Transformer, Spatio-Temporal Graph Neural Network. Across two spatial systems.
- 7 Evaluation → one look at the test set**  
November–December 2024 was never touched until every decision had been made.
- 8 Application → map, assistant, walking directions**  
A web app that turns the trained models into answers a person can act on.

#### Why the work was split into phases

Each phase had a written *Definition of Done* — a checklist that had to be satisfied before the next phase could start. Crucially, the checklists were verified **by scripts reading the actual output files**, not by me ticking boxes from memory. If a check said "the cleaned data is sorted by time", a script opened the file and checked all 79 million rows.

This sounds bureaucratic. It caught several serious bugs that would otherwise have silently corrupted everything downstream — including two that are documented in section 27.

## 4. Where the data came from

---

Four separate sources, each with its own quirks.

### 4.1 Citi Bike trip records — the core dataset

Citi Bike publishes every trip taken, monthly, as a CSV file. Two years is 90 files and about 15 GB uncompressed. Each row is one trip: a ride ID, start and end times, start and end station names and IDs, latitude and longitude for both ends, and whether the rider was a member or a casual user.

#### ENGINEERING DECISION

15 GB will not fit in this machine's 16 GB of RAM alongside anything else. Loading it with pandas — the obvious first instinct — would fail. Instead the files were read with **Polars** in streaming mode: select only the needed columns, cast types, and write straight out to Parquet in a single pass. The result is **2.5 GB** instead of 15 GB, and every later stage reads it lazily. The whole conversion took **2 minutes**.

Parquet matters here. It is a columnar format, so reading just the two columns you need doesn't touch the rest of the file, and it stores statistics per block that let a query skip whole sections. Everything downstream depends on that being fast.

### 4.2 Weather — hourly, from Open-Meteo

Fetches from the Open-Meteo historical archive: temperature, apparent temperature, humidity, precipitation, rain, snowfall, snow depth, wind speed and wind gusts, hourly, for the whole period. That is 17,544 rows — 731 days × 24 hours — with no gaps.

One variable, `visibility`, came back 100% empty. Rather than keeping a dead column that might later be mistaken for real data, it was removed from the request entirely.

### 4.3 Public events — NYC permitted events

The city publishes every permitted street event: block parties, film shoots, parades, construction closures. 557,674 rows covering 75,729 distinct events.

**The catch:** these records have no coordinates. They have free-text location descriptions like "Astoria Park: Soccer-01" or "5 Avenue between 42 Street and 59 Street". To use them spatially, 19,011 distinct location strings had to be geocoded — converted into latitude and longitude — which is covered in section 10.

### 4.4 Traffic speeds — and a false start worth recording

The first attempt used a CSV downloaded from the NYC Open Data web interface. It looked correct. It was not: the web export had been **silently capped**, and contained only five days of data from July 2018 — zero overlap with the 2023–24 project window.

## LESSON

A file that downloads successfully and opens cleanly can still be the wrong data. The only reason this was caught is that a sanity check compared the date range in the file against the project window. Always check that the data you have covers the period you think it does.

The fix was to pull directly from the **Socrata API** (dataset `i4gi-tjb9`), requesting only the columns that change per reading, and paginating *by time window* rather than by row offset — offset-based pagination on a large dataset gets slower and slower and can skip records if the underlying data shifts.

This returned **23,758,184 readings**. Completeness was then proved rather than assumed: before fetching, the server was asked for a `count(1)` over the same window. The two numbers matched exactly.

## 5. Cleaning it without hiding anything

Raw trip data contains records that are physically impossible: trips lasting three seconds, trips lasting four days, trips with no coordinates, trips ending before they started.

The temptation is to filter them out with a few conditions and move on. The problem with that is you never learn *how much* you threw away or *why*, and a bug in one condition can silently delete a large fraction of the data without anyone noticing.

### The waterfall approach

Instead, nine **named rules** are applied in a fixed order, and each removed row is attributed to the **first** rule it violates. The counts then sum exactly to the total removed — a self-checking arithmetic identity.

| Rule                   | Rows removed      | % of raw       | What it catches                   |
|------------------------|-------------------|----------------|-----------------------------------|
| duplicate_ride_id      | 0                 | 0%             | The same trip recorded twice      |
| null_ride_id           | 0                 | 0%             | A trip with no identifier         |
| null_timestamp         | 0                 | 0%             | Missing start or end time         |
| outside_project_window | 276               | 0.0003%        | Dated outside 2023–2024           |
| end_before_start       | 643               | 0.0008%        | Time travel                       |
| duration_too_short     | 182               | 0.0002%        | Under 60 s — a failed undock      |
| duration_too_long      | 37,627            | 0.047%         | Over 24 h — likely stolen or lost |
| missing_coords         | 206,222           | 0.260%         | No latitude/longitude             |
| coords_out_of_bbox     | 178               | 0.0002%        | Outside the New York area         |
| <b>Total removed</b>   | <b>245,128</b>    | <b>0.309%</b>  |                                   |
| <b>Kept</b>            | <b>79,165,067</b> | <b>99.691%</b> |                                   |

A quality report is generated *before* anything is deleted, so the decision to drop a category of record is made with the numbers visible.

Worth noting: **not a single duplicate ride ID in 79.4 million rows across two years**. The de-duplication step is kept anyway, so a future month can't slip one through unnoticed.

### Sorting 79 million rows by time

Everything downstream — lag features, chronological splits, the panel — assumes the data is in time order. The 90 source files arrive as separate months, so the combined dataset is *not* sorted globally.

### PROBLEM HIT

A single `ORDER BY started_at` across 79 million wide rows spilled **8 GB to disk** and had not finished after an hour.

**The fix** uses a property of the data: it is already partitioned by month. Sorting each month individually (about 3 million rows — comfortably in memory) and then concatenating the parts in month order produces the *identical* global ordering. Total time: about 90 seconds of sorting plus a streaming concatenation.

Rather than trusting that argument, the result is verified two independent ways: by reading the Parquet block statistics, and by scanning the entire timestamp column in file order looking for any inversion. Result: **0 violations, 0 inversions across 79,165,067 rows.**

## 6. The timezone problem nobody expects

This section covers a subtle issue that would have quietly corrupted the model, and is the kind of thing that separates a working pipeline from a broken one.

Citi Bike publishes times as **local New York wall-clock time with no timezone attached**. That seems harmless until daylight saving time.

### What goes wrong in autumn

On the first Sunday in November, clocks go back one hour. The local hour from 1:00am to 2:00am **happens twice**. Both passes are written to the file as "01:xx" with nothing to distinguish them.

If you bin those timestamps into 15-minute intervals using local time, both passes land in the same bins. You get one bin with roughly double the true demand, and the model learns a fictional annual spike.

### What goes wrong in spring

In March, clocks go forward. The local hour from 2:00am to 3:00am **does not exist**. Any timestamp in it is impossible. In practice a handful of records land there anyway, through clock skew — 617 rides in 2023, 582 in 2024.

### The fix

All timestamps are converted to **UTC**, which has no daylight saving and no repeated or missing hours, and stored that way. Local calendar features (hour of day, day of week) are derived by converting back when needed.

```
localized = expr.dt.replace_time_zone(tz, ambiguous="earliest", non_existent="null")
shifted   = (expr + duration(hours=1)).dt.replace_time_zone(tz, ...)
return coalesce(localized, shifted).dt.convert_time_zone("UTC")
```

For ambiguous times we take the earliest interpretation; for non-existent times we shift forward by an hour. This is deterministic and documented.

## Measuring the damage rather than assuming it

Because the earliest interpretation is used, the *second* pass through the repeated hour is unreachable. That means those UTC hours contain only half their true demand. Rather than hoping this doesn't matter, it was measured directly in the data:

| UTC hour         | Rides recorded | Interpretation                      |
|------------------|----------------|-------------------------------------|
| 2023-11-05 05:00 | 2,921          | Both passes collapsed into one hour |
| 2023-11-05 06:00 | 0              | The unreachable second pass         |
| 2024-11-03 05:00 | 3,534          | Same pattern, following year        |
| 2024-11-03 06:00 | 0              | Same                                |

Four affected hours were identified. Every row falling in them is flagged `dst_unreliable`, and every model excludes those rows from training. The models never see the artificial collapse, and the flag travels with the data so nobody downstream has to remember.

## 7. Turning trips into a grid of numbers

---

A model can't consume "a list of trips". It needs a rectangular table: for each place, at each time, how many bikes were taken and returned. Building that table is what this phase does.

### 7.1 Dividing the city into cells

We use **Uber's H3** system, which tiles the world in hexagons at a range of sizes. Hexagons are used rather than squares because every neighbour is the same distance away — with squares, diagonal neighbours are 41% further than edge neighbours, which distorts any neighbourhood calculation.

At resolution 8, a cell is about 0.74 km<sup>2</sup> — roughly a large city block cluster, containing about eight Citi Bike stations on average.

#### A 0.8-SECOND OPTIMISATION

Assigning 79 million trip endpoints to cells looks like 158 million geometric calculations. But Citi Bike *snaps every trip to a docking station*, so the whole dataset contains only **3,230 distinct coordinate pairs**. Index those 3,230 once into a lookup table and join. The operation takes **0.8 seconds** instead of the better part of an hour, and it is exact — not an approximation. Tests confirm the fast path and the slow path agree on real NYC points at three resolutions.

### 7.2 Deciding which cells count

1,570 cells were touched by at least one trip. Many of those are noise — a single tourist ride into an area with no service. Cells are kept only if they had **at least 500 trips and at least 180 active days** across the two years.

That keeps 1,483 cells at resolution 9, which carry **99.98% of all trip endpoints**. The rule discards 87 marginal cells and loses almost nothing.

### 7.3 Density is the contract

The panel has **exactly one row per active cell per 15-minute interval**, with no gaps. A cell with no trips in an interval gets a **0**, not a missing row.

This matters more than it sounds. Lag features ask "what was demand one hour ago?" — which is "four rows back" only if there is guaranteed to be a row for every interval. With gaps, "four rows back" could mean four hours back, and every feature would be silently wrong for some cells.

The grid is built as an explicit cross join of cells × timestamps, with counts joined onto it.

| Property       | Value                                                                                                                                                                     |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rows           | <b>104,071,008</b> = 1,483 cells × 70,176 intervals — exactly dense                                                                                                       |
| Time range     | 2023-01-01 05:00 UTC to 2025-01-01 04:45 UTC, 15-minute bins, zero irregular gaps                                                                                         |
| Columns        | <code>region_id</code> , <code>ts</code> , <code>pickups</code> , <code>dropoffs</code> , <code>net_flow</code> , <code>dst_unreliable</code> , plus eight target columns |
| Reconciliation | Panel pickups total 79,150,890 — matches trips in active cells exactly                                                                                                    |
| Sparsity       | 71.37% of cell-intervals have zero pickups at resolution 9                                                                                                                |
| Storage        | 277 MB across 24 monthly partitions                                                                                                                                       |

## 7.4 What the model is asked to predict

Four horizons: 15, 30, 45 and 60 minutes ahead, for both pickups and dropoffs — eight targets in total. Each is computed by looking forward in the sorted series.

These targets are **not trusted, they are tested**. A separate piece of code re-derives each target using an independent timestamp self-join and asserts the two agree — in unit tests on a small sample, and in the phase gate on 60 sampled cells covering 4.2 million rows.

### A DELIBERATE CHOICE ABOUT THE END OF THE SERIES

For the last few intervals there is no future to look at. Those targets are set to **NULL, never 0**. A zero would teach the model that demand collapses at the end of every series — a phantom pattern that doesn't exist. The count of NULLs is verified to be exactly cells × horizon.

## 8. The baseline that had to be beaten

Before building anything sophisticated, you need to know what "good" means. The baseline here is **Seasonal Naive**: to predict demand at a given time, just look up what happened at the same time in a previous cycle.

Two variants were tested: same time yesterday, and same time last week.

### THE SUBTLE DETAIL THAT MAKES THIS BASELINE HONEST

To forecast the interval at `t + h`, you need the value one season before *that instant* — which is `lag(season - h)`, not `lag(season)`.

For one-day seasonality at a 15-minute horizon: **lag 95, not lag 96**.

Using lag 96 would misalign the baseline by up to an hour and hand the real model an unearned victory. Eight extra lag columns are materialised specifically for the baseline and deliberately kept out of the model's feature list.

## Results of the gate

Phase 3 was a **stop-and-verify gate**: if LightGBM couldn't beat Seasonal Naive on every one of the eight targets, the project stopped and the problem got diagnosed. It passed 8 out of 8, on validation data (September–October 2024, 8.68 million rows).

| Target          | Naive MAE | LightGBM MAE | Improvement | Hotspot F1 |
|-----------------|-----------|--------------|-------------|------------|
| pickup, 15 min  | 0.8768    | 0.6945       | 20.8%       | 0.602      |
| pickup, 30 min  | 0.8768    | 0.7013       | 20.0%       | 0.600      |
| pickup, 45 min  | 0.8768    | 0.7049       | 19.6%       | 0.598      |
| pickup, 60 min  | 0.8768    | 0.7076       | 19.3%       | 0.598      |
| dropoff, 15 min | 0.8673    | 0.6920       | 20.2%       | 0.595      |
| dropoff, 30 min | 0.8673    | 0.6984       | 19.5%       | 0.592      |
| dropoff, 45 min | 0.8673    | 0.7035       | 18.9%       | 0.590      |
| dropoff, 60 min | 0.8672    | 0.7072       | 18.5%       | 0.589      |

Between the two naive variants, **same-week beat same-day** (0.872 vs 0.899 mean MAE) on validation — weekly rhythm matters more than daily rhythm for commuter cycling. Section 17 describes how this finding reversed on the test set, and why.

## How the data is split

Splits are **chronological**, never random. Randomly shuffling time-series data lets the model see the future while training, which produces excellent scores and a useless model.

| Split    | Period              | Purpose                              |
|----------|---------------------|--------------------------------------|
| Train    | Jan 2023 – Aug 2024 | Fitting the models                   |
| Validate | Sep – Oct 2024      | Every comparison and choice          |
| Test     | Nov – Dec 2024      | Opened exactly once, at the very end |

## 9. Building features, and proving they don't cheat

A feature is a number describing the situation at forecast time. The model sees the features and predicts the target. The single most dangerous mistake in this kind of project is **data leakage** — accidentally giving the model information from the future.

Leakage is dangerous because it doesn't look like a bug. It looks like success. Your accuracy improves, you ship the model, and it fails completely in production because that information isn't available in real life.

### 9.1 Pinning down exactly what "now" means

Everything depends on defining forecast time precisely. Consider the 15-minute bin from 17:00 to 17:15. If you make a forecast "at" that bin, are you standing at 17:00 or 17:15?

The project pins it down: **forecast time is the END of the bin**. This has two consequences, both good:

- A one-step-ahead forecast is genuinely 15 minutes ahead. Under the other reading it would be somewhere between 15 and 30 minutes, and every horizon label in the project would be off by one.
- The current bin's demand is a *legal input* — the bin has completed. This is the single most informative feature available, and the earlier phase had omitted it.

This decision is written down in `docs/FORECAST_TIME_CONVENTION.md` so it can't drift.

### 9.2 The eight feature families

| #     | Family              | Columns | What it captures                                                                                                                 |
|-------|---------------------|---------|----------------------------------------------------------------------------------------------------------------------------------|
| A     | Recent demand       | 20      | Demand in this cell 1, 2, 4, 8, 12, 24, 96, 192 and 672 intervals ago, plus the just-completed bin                               |
| B     | Calendar & seasonal | 55      | Time of day and day of week as smooth cyclical values, holidays, rush-hour flags, and demand at the same slot in previous cycles |
| C     | Rolling trends      | 49      | Moving averages, sums, standard deviations and exponentially-weighted means over several windows; short-term change              |
| D     | Weather             | 17      | Temperature, rain, snow, wind, plus derived flags and short-term temperature change                                              |
| E     | Events              | 6       | Count and scale of permitted events active in this cell                                                                          |
| F     | Traffic             | 7       | Average speed and congestion on road sensors near this cell                                                                      |
| G     | Neighbours          | 9       | Demand in the six surrounding cells; this cell relative to its neighbourhood                                                     |
| H     | Station network     | 4       | How many stations are in the cell and how busy they historically are                                                             |
| Total |                     | 168     |                                                                                                                                  |

### 9.3 How leakage is prevented — and proved

Every rolling and lag feature is computed with an explicit `PARTITION BY region_id` over a series sorted by time, so a cell can only ever see its own past.

But writing the code carefully isn't proof. **Fifteen dedicated tests re-derive every lag feature using a completely independent method** — a timestamp self-join, written separately from the production code — and assert the two agree exactly. Twenty more tests cover the advanced families.

**WHY RE-DERIVATION RATHER THAN INSPECTION**

Reading code to check for leakage catches the bugs you thought of. Re-deriving the answer a different way catches the ones you didn't. If a window is off by one, or a sort is wrong, or a partition is missing, the two methods disagree and the test fails.

### 9.4 Making 104 million rows fit in 16 GB

Computing 168 features over 104 million rows in one pass spilled 12 GB to disk and threatened to fill the drive.

The fix uses a mathematical property: because every lag and rolling window is partitioned by cell, **processing a batch of cells at a time is exactly equivalent** to processing all of them together. It is not an approximation. Nine batches of about 12 million rows finished in **6.4 minutes with 4 KB of spill**.

## 10. Which features actually earned their place

Having 168 features doesn't mean they all help. More features means slower training, more memory, more pipeline complexity to maintain — and possibly worse accuracy, because a model with a fixed capacity spends some of it on noise.

So the families were tested by **ablation**: start with family A alone, add B, measure; add C, measure; and so on. Every step gets an identical training budget and identical data, so the only thing changing is the feature set.

### Results at H3 resolution 8

| Step | Family added        | Features | Pickup MAE | Dropoff MAE | Change | Verdict        |
|------|---------------------|----------|------------|-------------|--------|----------------|
| A    | Recent demand       | 21       | 1.6944     | 1.6675      | —      | baseline       |
| B    | + Calendar/seasonal | 76       | 1.5791     | 1.5606      | +6.60% | Helps a lot    |
| C    | + Rolling trends    | 125      | 1.5695     | 1.5503      | +0.64% | Helps          |
| D    | + Weather           | 142      | 1.5703     | 1.5511      | -0.05% | No benefit     |
| E    | + Events            | 148      | 1.5702     | 1.5516      | -0.01% | No effect      |
| F    | + Traffic           | 155      | 1.5702     | 1.5531      | -0.05% | No effect      |
| G    | + Neighbours        | 164      | 1.5675     | 1.5464      | +0.30% | Helps slightly |
| H    | + Station network   | 168      | 1.5684     | 1.5475      | -0.07% | No benefit     |

**The chosen feature set is A + B + C: 125 features.**

## What this actually means

The headline finding is blunt: **weather, events and traffic contribute essentially nothing**. Nearly all the predictable signal beyond recent demand is *calendar and seasonality* — which makes sense for commuter cycling, and is cheap to compute.

Three honest caveats bound how far this generalises:

1. **"Hurts" should be read as "no measurable benefit", not "actively harmful"**. Every step gets the same fixed training budget, so adding 17 weather columns dilutes a fixed capacity. A  $-0.05\%$  move at this scale is dilution, not evidence that rain is anti-informative.
2. **Events carry a 25% geocoding hole**. 74.7% of event locations resolved to coordinates; the rest were free-text descriptions the geocoder couldn't place, or were rejected by a borough cross-check. A better geocoder might change family E — though at  $\pm 0.01\%$  it would have to change it enormously.
3. **The traffic result was initially suspect, and was retested**. This is covered immediately below, because it is the most interesting thing in the section.

## Giving traffic a fair second chance

The first ablation ran at H3 resolution 9, where the traffic sensor network reaches only **76 of 1,483 cells — 5.1%**. The feature is null for 95% of rows. That isn't a test of "does traffic help"; it's a test of "does traffic help in 5% of Manhattan".

When resolution 8 was later chosen, traffic coverage rose to **20.2% of cells — four times better**. The entire ablation was re-run to give families D–H a fair trial rather than inheriting a verdict from a grid where the data barely existed.

### THE RESULT

**Quadrupling the coverage changed nothing.** Traffic still contributes  $-0.05\%$ . That turns a weak result ("traffic didn't help, but coverage was thin") into a defensible negative finding: the feature was given four times the reach and still earned nothing.

Family G (neighbours) was the one genuine change, moving from  $-0.06\%$  at resolution 9 to  $+0.30\%$  at resolution 8. It was still not adopted, because the ablation is cumulative — taking G would drag D, E and F along with it for a gain inside the noise.

## What the model actually leans on

Gain-based feature importance from the final trained model confirms the ablation independently:

| Rank | Feature                                  | Family | What it is                                                    |
|------|------------------------------------------|--------|---------------------------------------------------------------|
| 1    | <code>pickups_ewm_4</code>               | C      | Exponentially-weighted mean of recent pickups                 |
| 2    | <code>pickups_roll_mean_4</code>         | C      | Average pickups over the last hour                            |
| 3    | <code>pickups_lag_0</code>               | A      | The just-completed interval                                   |
| 4    | <code>pickups_slot_expanding_mean</code> | B      | This cell's historical average for this weekday and time slot |
| 5    | <code>pickups_roll_sum_4</code>          | C      | Total pickups over the last hour                              |

The top five features come from exactly the three families the ablation kept. Two independent methods agreeing is a good sign that the conclusion is real.

## 11. Choosing the size of a map cell

This is one of the two headline research results, and the interesting part is not the answer but the fact that **the obvious way to measure it is wrong**.

Three cell sizes were tested — H3 resolutions 8, 9 and 10 — with the model, features, dates, training budget and metrics held identical. Only the geometry changed.

| Resolution | Cells | Area (km <sup>2</sup> ) | % zero demand | MAE          | WAPE         | Hotspot F1   |
|------------|-------|-------------------------|---------------|--------------|--------------|--------------|
| H3-8       | 321   | 0.741                   | 47.5%         | 1.654        | <b>0.308</b> | <b>0.748</b> |
| H3-9       | 1,483 | 0.106                   | 71.4%         | 0.705        | 0.605        | 0.590        |
| H3-10      | 2,162 | 0.015                   | 75.6%         | <b>0.580</b> | 0.731        | 0.481        |

### Why these numbers can't be compared directly

Look at the table again. **MAE says resolution 10 is best. WAPE and Hotspot F1 say resolution 8 is best.** They disagree completely. That total disagreement is the clue that none of them is measuring what we want.

- **MAE falls automatically as cells shrink.** Smaller cells hold smaller counts, so absolute errors shrink mechanically. In the limit, a cell so small it always contains zero bikes has near-perfect MAE and zero usefulness.
- **WAPE falls automatically as cells grow.** Aggregation smooths relative error. In the limit, a single cell covering all of New York has excellent WAPE and tells you nothing.
- **Hotspot F1 depends on how many cells exist.** Identifying the top 10% of 321 cells is a genuinely different task from the top 10% of 2,162.

### The fix: a skill score

The solution is to measure each resolution against *its own* naive baseline:

$$\text{skill} = 1 - (\text{model MAE} / \text{Seasonal Naive MAE})$$

Both numbers are computed on the same grid, so whatever scale effect inflates or deflates one does the same to the other, and it cancels out. What remains is how much the model actually *knows* beyond the obvious.

| Resolution | Model MAE | Naive MAE | Skill         | Reading                 |
|------------|-----------|-----------|---------------|-------------------------|
| H3-8       | 1.654     | 2.224     | <b>0.2563</b> | Best — and least sparse |
| H3-9       | 0.705     | 0.873     | 0.1920        |                         |
| H3-10      | 0.580     | 0.706     | 0.1791        | Lowest MAE, least skill |

**Resolution 8 wins** — and it also wins on the secondary criterion of sparsity, at 47.5% empty cells against 75.6% at resolution 10. A grid where three-quarters of cells are empty at any moment is a hard learning problem and a

hard thing to act on.

THE TRANSFERABLE LESSON

When comparing models across different data granularities, most accuracy metrics are contaminated by the granularity itself. Always compare against a baseline computed on the same granularity. If your metrics disagree with each other, that's information — it usually means they're measuring the setup rather than the skill.

12. Hexagons versus squares

H3 uses hexagons; Google's S2 uses quadrilaterals. Does the shape matter?

To answer fairly, you need an S2 level whose cells are comparable in size to H3-8. Guessing would invalidate the comparison, so the level was **selected by a matching procedure**: compare candidate levels on median cell area *and* active-cell count, using a weighted log-ratio distance.

Log-ratio matters: it makes "twice as coarse" and "twice as fine" cost the same. On a linear scale, being too coarse looks cheaper than being too fine, which would bias the choice.

**S2 level 13 was selected**, scoring 0.319 against 0.951 for the next-best candidate — a clear separation, not a close call.

| System | Cells | Area (km <sup>2</sup> ) | % zero | MAE   | Skill         | Hotspot F1   | Features size | Train time  |
|--------|-------|-------------------------|--------|-------|---------------|--------------|---------------|-------------|
| H3-8   | 321   | 0.741                   | 47.5%  | 1.654 | 0.2563        | 0.748        | 430 MB        | 32 s        |
| S2-13  | 231   | 1.088                   | 42.1%  | 2.002 | <b>0.2741</b> | <b>0.769</b> | <b>328 MB</b> | <b>24 s</b> |

The honest reading

S2-13 edges H3-8 on skill, 0.274 to 0.256. But **the honest conclusion is that the spatial system barely matters**, for a specific reason: S2-13's cells are still **47% larger** than H3-8's.

Matched granularity is not identical granularity — quadrilaterals simply cannot tile to the same areas as hexagons. That residual coarseness plausibly accounts for the entire 1.8-point gap.

**Resolution is the decision that matters. Hexagons versus squares is close to a coin flip**, with S2 marginally cheaper to build and store. This result was later confirmed independently at the final evaluation, where S2-13 scored higher skill for all three model families — exactly as this sweep predicted.

## 13. LightGBM, tuned properly

LightGBM is a gradient-boosted decision tree model. It builds hundreds of small decision trees in sequence, each one correcting the errors of the ones before it. It is the standard strong baseline for tabular problems, and it is fast on a CPU.

### Three things fixed from the earlier phase

#### THE OBJECTIVE WAS TESTED, NOT ASSUMED

Bike demand is a *count* — a non-negative integer, often zero. The Poisson objective is designed for counts; standard regression is not. Rather than assuming Poisson was better, both were trained and compared on validation:

| Objective       | Validation MAE | Result      |
|-----------------|----------------|-------------|
| Poisson         | <b>1.5695</b>  | <b>Kept</b> |
| Regression (L1) | 1.5739         | Discarded   |

Poisson won, but only by 0.3%. Worth knowing — the theoretical argument is much stronger than the empirical gap.

#### THE ROUND CAP WAS ACTUALLY BINDING

In the earlier phase, every model hit the 500-round training cap without early stopping ever triggering. That means the models were *still improving* when training was cut off, and those numbers were a floor, not the model's ceiling.

The cap was raised to 2,000 and early stopping was allowed to decide. The final models stopped between **569 and 1,101 rounds** — genuinely converged.

#### A BOUNDED HYPERPARAMETER SEARCH

A 10-trial Optuna search over tree depth, leaf count, learning rate, minimum samples per leaf, feature and row sampling fractions, and two regularisation terms. Validation MAE improved from 1.5695 to **1.5544**.

The search was deliberately small. The ablation had already shown that the *feature set* matters far more than hyperparameters here, so an exhaustive sweep would burn hours to move the third decimal place.

## Final model summary

| Property            | H3-8                       | S2-13           |
|---------------------|----------------------------|-----------------|
| Features            | 125 (families A+B+C)       | 125 (identical) |
| Objective           | Poisson                    | Poisson         |
| Training rows       | 3,757,626                  | 2,704,086       |
| Validation rows     | 1,879,776                  | 1,352,736       |
| Models trained      | 8 (2 targets × 4 horizons) | 8               |
| Mean validation MAE | 1.5743                     | 1.9007          |
| Total training time | 48 minutes on CPU          | 23 minutes      |

## 14. The two deep-learning models

Two neural networks were built to test whether deep learning beats gradient boosting on this problem. Both were written by hand rather than pulled from a library, so their internals are inspectable and their inputs are exactly controlled.

### 14.1 Temporal Fusion Transformer (TFT)

A TFT is designed for forecasting when you have three different kinds of input:

- **Static** — things that never change about a cell: its area, its station count, its historical volume.
- **Known future** — things you know in advance for the time you're forecasting: it will be 5pm, it will be a Wednesday, it will be a holiday. This is not leakage; a calendar is genuinely known ahead of time.
- **Observed past** — things you only know up to now: recent demand, recent weather.

The architecture uses *gated residual networks* to let the model skip transformations it doesn't need, a *variable selection* layer that learns which inputs matter, an LSTM encoder over the last 96 intervals (24 hours), and an interpretable multi-head attention layer. Output is non-negative by construction. 239,688 parameters.

### 14.2 Spatio-Temporal Graph Neural Network (ST-GNN)

The idea here is that **cells are not independent**. If demand surges in one cell, neighbouring cells often follow. A graph neural network can model that directly.

The graph has one node per cell and an edge between cells that physically touch — first-ring adjacency, both directions. Two graph-attention layers let each cell gather information from its neighbours, weighted by learned importance. A GRU then handles the time dimension over a 24-interval (6-hour) window, plus same-time-yesterday and same-time-last-week snapshots.

The graph attention is implemented with an edge-list and a segment softmax via `index_add_`, which avoids building a dense adjacency matrix. Only **35,432 parameters** — around one-seventh of the TFT.

## 14.3 The fairness rule

### MAKING THE COMPARISON MEAN SOMETHING

All three model families get access to the same *categories* of information: past demand, calendar, weather, spatial structure, network statistics. No model receives privileged future information. How each architecture represents that information internally is allowed to differ — that's the point of comparing architectures — but the information itself is held constant.

Scaling statistics are fitted on the **training split only** and baked into the exported data bundle, so it is structurally impossible for the deep models to normalise using validation or test statistics.

## 15. Training on a free GPU

The laptop has an RTX 3050 Ti with 4 GB of VRAM, and the installed PyTorch was a CPU-only build. Training both deep models locally was estimated at several hours.

Instead the deep half moved to **Kaggle**, which provides free Tesla T4 GPUs. The LightGBM, DuckDB and Polars stages stayed local, because they are CPU- and disk-bound and need the 5.9 GB feature table.

### What was uploaded

Only what the neural networks actually need: a compact tensor bundle per spatial system.

|                 |         |           |                                                  |
|-----------------|---------|-----------|--------------------------------------------------|
| demand.npy      | int16   | [T, N, 2] | pickups + dropoffs per interval per cell         |
| calendar.npy    | float32 | [T, 8]    | known-future time features                       |
| weather.npy     | float32 | [T, 6]    | observed at forecast time, already lagged        |
| edges.npy       | int32   | [2, E]    | cell adjacency, both directions                  |
| node_static.npy | float32 | [N, 5]    | per-cell constants                               |
| dst_mask.npy    | bool    | [T]       | the DST-distorted intervals, excluded            |
| meta.json       |         |           | cell IDs, split boundaries, TRAIN-fitted scaling |

Total 557 MB for three bundles.

### A PRECAUTION WORTH COPYING

The model definitions file `src/models/deep.py` was **shipped with the data** rather than retyped into the notebook. If the Kaggle copy of the architecture had drifted from the repository copy, the GPU results would not correspond to anything reproducible locally — and nobody would notice. The notebook *refuses to run* if that file is missing, rather than silently falling back.

## Results after 8 epochs (~23 GPU-minutes total)

| Model  | Grid  | Parameters | Best validation MAE | Converged?           |
|--------|-------|------------|---------------------|----------------------|
| TFT    | H3-8  | 239,688    | <b>1.6716</b>       | No — still improving |
| ST-GNN | H3-8  | 35,432     | 1.7646              | No — still improving |
| TFT    | S2-13 | 239,688    | 1.9674              | No — still improving |
| ST-GNN | S2-13 | 35,432     | 2.1988              | No — still improving |

These validation figures average all four horizons and both targets, so they are *not* directly comparable to the horizon-specific LightGBM numbers elsewhere. Section 16 is the like-for-like comparison.

### A BUG FOUND IN THE EVALUATION LOOP

Validation was originally scored on the *first N anchors* of the validation split. The split begins at midnight on 1 September, so this evaluated the model almost entirely on empty overnight intervals. It reported an MAE of 0.665 where the true figure was 1.271 — nearly twice as good as reality.

The fix samples anchors at a stride across the whole validation period. It was applied in both the local script and the Kaggle notebook, so the two cannot diverge.

## 16. The final exam: the test set

Everything so far was decided on the validation split. November and December 2024 had never been looked at. Phase 7 opened it **exactly once**.

This discipline is the reason the numbers below can be trusted. If you evaluate on the test set repeatedly and tune based on what you see, it silently becomes a second validation set and the reported performance becomes optimistic.

### 16.1 Results at H3-8 (1,877,208 test rows)

| Model                      | Pickup MAE | Dropoff MAE | WAPE   | RMSE   | Hotspot F1 |
|----------------------------|------------|-------------|--------|--------|------------|
| LightGBM (tuned)           | 1.1408     | 1.1275      | 0.3573 | 2.2020 | 0.7248     |
| ST-GNN                     | 1.2024     | 1.1978      | 0.3686 | 2.4180 | 0.7260     |
| TFT                        | 1.2919     | 1.4969      | 0.3523 | 2.5071 | —          |
| Seasonal Naive (yesterday) | 1.8430     | 1.8253      | 0.5771 | 4.1931 | 0.6265     |
| Seasonal Naive (last week) | 1.9575     | 1.9409      | 0.6130 | 4.4710 | 0.6380     |

All three real models beat both naive baselines by 35–42%. Each one is a genuine model rather than a dressed-up persistence rule.

### 16.2 How accuracy degrades with horizon

| Horizon    | Pickup MAE | Dropoff MAE | WAPE   | Hotspot F1 |
|------------|------------|-------------|--------|------------|
| 15 minutes | 1.1408     | 1.1275      | 0.3573 | 0.7248     |
| 30 minutes | 1.1696     | 1.1549      | 0.3663 | 0.7224     |
| 45 minutes | 1.1879     | 1.1739      | 0.3720 | 0.7220     |
| 60 minutes | 1.2088     | 1.1934      | 0.3786 | 0.7216     |

Degradation is mild and monotone — a one-hour forecast is only **6% worse** than a 15-minute one. For an operator planning a truck route, that is the difference between a useful tool and a useless one.

### 16.3 The complete six-configuration matrix

Raw MAE is not comparable across spatial systems, for the reasons in section 11. Skill against Seasonal Naive on the same grid is.

| Configuration           | MAE           | WAPE          | Hotspot F1    | Skill vs naive |
|-------------------------|---------------|---------------|---------------|----------------|
| <b>LightGBM @ S2-13</b> | 1.3777        | <b>0.3105</b> | <b>0.7485</b> | <b>0.4127</b>  |
| LightGBM @ H3-8         | <b>1.1408</b> | 0.3573        | 0.7248        | 0.3810         |
| TFT @ S2-13             | 1.4762        | 0.3569        | —             | 0.3707         |
| ST-GNN @ S2-13          | 1.4817        | 0.3269        | 0.7449        | 0.3683         |
| ST-GNN @ H3-8           | 1.2024        | 0.3686        | 0.7260        | 0.3476         |
| TFT @ H3-8              | 1.2919        | 0.3523        | —             | 0.2990         |

## 16.4 Answering RQ3 carefully

**LightGBM is first at both spatial systems.** That is the answer. But two details deserve care, because the first one nearly produced a wrong conclusion.

### THE TWO DEEP-LEARNING "WINS" THAT WEREN'T

At H3-8, two results looked like genuine architecture effects:

- **TFT won WAPE** (0.3523 vs 1.1408's 0.3573). WAPE weights by volume, so this suggested TFT was better on the busy cells that matter operationally.
- **ST-GNN won Hotspot F1** (0.7260 vs 0.7248). Hotspot F1 ranks cells against each other, which is precisely what a graph's adjacency structure exists to inform.

Both were plausible. Both landed exactly where the architecture would predict. And both **reversed at S2-13**, where LightGBM took MAE, WAPE and Hotspot F1 outright.

#### WHY THIS MATTERS

Two sub-1% margins that flip under a change of tiling scheme are **noise, not evidence**. Had the project stopped at H3-8, the write-up would have claimed a nuanced three-way split — a more interesting story that happens to be wrong.

Both results are still reported, because suppressing them would be selective. They are simply labelled as what they are.

### THE CAVEAT ABOUT TRAINING BUDGET

The deep models were stopped at 8 epochs, and **all four were still improving on their final epoch** — the TFT at S2-13 dropped 7% on its last epoch alone. LightGBM, by contrast, ran to early-stopping convergence.

So the honest statement is: "**a tuned LightGBM beats an 8-epoch TFT and ST-GNN**", not "gradient boosting beats deep learning on this problem". LightGBM is clearly the better *value* here — it converged in 48 minutes of CPU time and won — but the deep models were not given their best shot.

## 17. Where the model gets things wrong

An average error hides more than it reveals. This section breaks the error down to find where the model actually fails.

### 17.1 By time of day

| Local hour | MAE   | Mean actual demand | Comment                            |
|------------|-------|--------------------|------------------------------------|
| 16:00      | 1.809 | 6.07               | Worst hour of the day              |
| 17:00      | 1.768 | 6.44               | Evening commute peak               |
| 15:00      | 1.699 | 5.69               |                                    |
| 02:00      | 0.348 | 0.36               |                                    |
| 03:00      | 0.281 | 0.26               | Best hour — almost nothing happens |

Error tracks demand level almost exactly, which is expected for a count process.

### 17.2 By demand level — the sharpest result

| Demand band  | MAE   | Mean actual | Relative error |
|--------------|-------|-------------|----------------|
| Low third    | 0.404 | 0.00        | —              |
| Middle third | 0.837 | 1.35        | 62%            |
| High third   | 2.746 | 10.65       | 26%            |

**Absolute error concentrates in busy cells; relative error is more than twice as bad in quiet ones.** Which metric you quote decides which cells look broken. Both are true, and reporting only one would be misleading.

### 17.3 Rain — a result that is easy to misread

| Condition | MAE   | Mean actual demand | Rows      |
|-----------|-------|--------------------|-----------|
| Dry       | 1.177 | 3.40               | 1,665,027 |
| Raining   | 0.853 | 1.57               | 206,724   |

**DO NOT READ THIS AS "THE MODEL IS BETTER IN THE RAIN"**

Mean demand is 1.57 in rain against 3.40 in dry conditions. **Rain suppresses cycling**, and a smaller count carries a smaller absolute error. The apparent improvement is a demand effect, not a skill effect.

This is exactly the kind of number that gets quoted out of context as a model strength.

## 17.4 The 100 largest errors

Pulling the single worst 100 predictions reveals a strikingly concentrated pattern:

- **73% are under-forecasts** — mean actual 78.6 against mean predicted 57.7
- **38 of 100 occur at 16:00**, and 18 more at 18:00
- They fall in just **22 distinct cells** out of 321

So the single characteristic failure is: **the model under-predicts afternoon-rush spikes in a small set of very high-volume cells.**

That is also the most operationally expensive place to be wrong, and it is a concrete target for future work — a spike-aware loss function, quantile prediction heads, or per-cell capacity terms.

## 17.5 A baseline result that flipped

On validation, same-week Seasonal Naive beat same-day (0.872 vs 0.899), and it was adopted as the primary baseline. **On the test set the order reverses:** same-day scores 1.8430 against same-week's 1.9575.

The explanation is the calendar. November and December contain Thanksgiving and Christmas, which break week-over-week patterns far more than they break day-over-day patterns. Both variants are reported rather than quietly keeping the flattering one.

## 18. Estimating how many docks exist

To answer "will there be a bike", you need to know how many docks a station has and how many bikes are currently in them. **Neither number exists in any public dataset for 2023–24.**

Citi Bike publishes trips. It does not publish historical dock occupancy.

### Estimating capacity from flow

The approach infers capacity from behaviour. For each station and each day, compute the running cumulative net flow — dropoffs minus pickups. The *range* of that curve is a lower bound on the station's capacity: a station whose inventory swung by 30 bikes must hold at least 30 docks.

Taking the 95th percentile of that daily range across two years gives a stable estimate.

| Measure                           | Value                           |
|-----------------------------------|---------------------------------|
| Station-days analysed             | 1,496,558 across 2,459 stations |
| Median estimated capacity         | 15 docks                        |
| 10th / 90th percentile            | 8 / 53 docks                    |
| Sized from own flow               | 2,387 stations                  |
| Fell back to volume-decile median | 72 low-volume stations          |

## Reconstructing daily inventory

For each station-day, the starting inventory must lie in a feasible interval — high enough that the station never goes negative, low enough that it never overflows. Within that interval, the point closest to 50% of capacity is chosen.

Inventory is **reset every day rather than chained across two years**, because errors would otherwise compound into nonsense by month three.

**1.73% of station-days have no feasible starting inventory at all** — meaning staff must have intervened that day. Those are flagged `inventory_unreliable` rather than forced to fit. Median fit error across all days: 0.00.

### LABELLED AS AN APPROXIMATION EVERYWHERE

Every artifact, API response and UI element carrying an inventory number also carries a flag saying it is estimated. The word "capacity" here means "a statistical estimate derived from observed flow", and the code says so in its own docstring.

## 19. Moving bikes around, and why it didn't help

The final research question: does a better forecast produce better operations?

### The simulation

Fourteen days of the test period, 1,348 intervals × 321 cells. A greedy rebalancer runs every hour: find cells projected to fall below a safety threshold, find cells above a target level, and move bikes from the latter to the former, nearest first, respecting capacity limits.

Four forecast sources drive it — perfect foresight (the actual future), LightGBM, Seasonal Naive, and doing nothing — and **all are scored against the identical actual demand**.

| Forecast              | Safety | Service level | Unserved     | Overflow     | Bikes moved | Moves |
|-----------------------|--------|---------------|--------------|--------------|-------------|-------|
| <b>No rebalancing</b> | —      | <b>0.9958</b> | <b>8,540</b> | <b>6,286</b> | 0           | 0     |
| Perfect foresight     | 5%     | 0.9940        | 12,294       | 11,432       | 9,286       | 2,615 |
| <b>LightGBM</b>       | 5%     | 0.9942        | 11,904       | 10,782       | 7,720       | 2,107 |
| Seasonal Naive        | 5%     | 0.9939        | 12,391       | 11,355       | 9,531       | 2,699 |
| Perfect foresight     | 10%    | 0.9891        | 22,297       | 21,475       | 23,383      | 6,305 |
| LightGBM              | 10%    | 0.9898        | 20,957       | 19,969       | 20,743      | 5,814 |
| Seasonal Naive        | 10%    | 0.9891        | 22,235       | 21,291       | 23,468      | 6,402 |

### Answer (a): among forecasts, yes — and surprisingly

At the 5% safety threshold, LightGBM leaves **11,904** pickups unserved, against perfect foresight's **12,294** and Seasonal Naive's **12,391**.

### A real model beat a perfect forecast.

That is not a paradox once you see the mechanism. LightGBM systematically under-forecasts — 73% of its largest errors are under-predictions, from section 17. So it triggers fewer interventions: **2,107 moves instead of 2,615**. In a regime where moving is harmful, restraint wins.

The phase specification had warned that "a model with slightly worse MAE can still win operationally". The reality turned out sharper: *even the perfect forecast is not the best rebalancer*.

### Answer (b): rebalancing itself does not pay here

Every intervention makes things worse. Unserved demand rises from 8,540 to 11,904; overflow rises from 6,286 to 10,782.

The cause is **granularity, not the algorithm**. At H3-8, 321 cells aggregate 2,459 stations — roughly 7.7 stations and about 115 docks per cell — against a median of about 3 trips per 15-minute interval.

### THE STRUCTURAL REASON

**A region that large essentially cannot run dry.** That is why doing nothing already serves 99.58% of demand. There is almost no shortage to recover, so the greedy rule spends its moves stripping bikes from cells above the target (which need them later) to top up cells below the safety floor (which are empty because they have no demand).

The defensible conclusion: **rebalancing value cannot be demonstrated at region granularity with estimated capacity.** The bike-rebalancing problem is real, but it lives at *station* level, and proving it needs dock occupancy data this project does not have.

Reporting a positive result here would have required tuning the thresholds until the sign flipped — which is exactly what this phase was written to prevent.

## 20. The prediction interface

All the research output is reduced to a single function, so a UI, a batch job and a test all share exactly one code path:

```
predict(spatial_system, model_name, forecast_time, horizon) →  
    region_id, forecast_time, horizon,  
    predicted_pickups, predicted_dropoffs,  
    projected_inventory, shortage, surplus
```

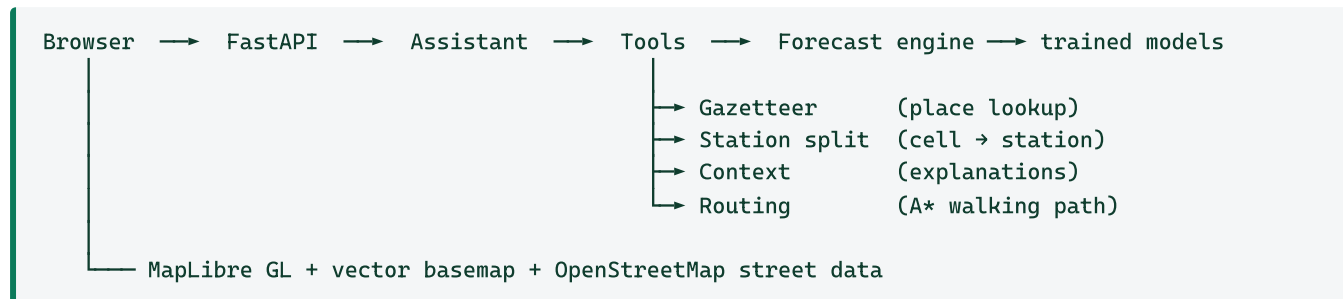
Design decisions it enforces:

- **It refuses a forecast time that isn't on the 15-minute grid**, rather than silently rounding. Rounding would move the target without telling the caller.
- **Projected inventory accumulates over horizons 1..h.** Asking for the one-hour horizon runs all four and sums them, because a cell that is fine now but empties within the hour should surface *now*.
- **Every row carries `inventory_estimated`**, so a caller cannot mistake the operational columns for measurements.

Ten tests cover the contract and the end-to-end path — including that shortage and surplus follow arithmetically from projected inventory, and that no cell is ever simultaneously short and in surplus.

## 21. Architecture of the app

The research half produces numbers. The application half makes them usable: you type "will I find a bike near Times Square?" and get a real answer from the trained models.



Every API endpoint is a thin shim over the same tool functions the assistant calls. This is deliberate: **the map and the chat cannot disagree with each other**, because they are reading the same numbers from the same functions.

### The forecast engine

One entry point serving all five models. It caches boosters, tensor bundles and cell mappings in memory, because the original interface re-read them on every call.

The deep models needed a separate serving path, and for a real reason: LightGBM consumes *one row* of the feature table, while ST-GNN and TFT consume a *window* of history as a tensor — 24 intervals for the graph model, 96 for the transformer — plus same-time-yesterday and same-time-last-week slices. These genuinely differ, so they are kept separate rather than forced into one signature.

| Model          | Response time            | Predicted citywide pickups |
|----------------|--------------------------|----------------------------|
| LightGBM       | 1.60 s cold, ~0.3 s warm | <b>2,862</b>               |
| ST-GNN         | 0.17 s                   | 3,001                      |
| TFT            | 0.40 s                   | 2,585                      |
| Seasonal Naive | 0.03 s                   | 2,727                      |
| <b>Actual</b>  | —                        | <b>2,878</b>               |

LightGBM's 2,862 against an actual 2,878 is a good illustration of the test-set result in a single live query.

## 22. From a cell forecast to a station answer

Here is a genuine mismatch between what the models do and what users ask.

**The models forecast cells.** A user asks about a *station*. An H3-8 cell is about 0.74 km<sup>2</sup> and contains roughly eight Citi Bike stations. There is no per-station model anywhere in this project.

## The disaggregation

Each station's share of its cell's forecast is its **historical share of that cell's trips**, taken from the station registry over two full years. A station that historically took 18% of its cell's pickups is assigned 18% of the cell's forecast.

For example, near Times Square: Broadway & W 48 St takes 7.4% of its cell, W 47 St & 6 Ave 5.9%, 6 Ave & W 45 St 10.4%. Shares sum to exactly 1.000 for all 321 cells — verified.

### STATED AS AN APPROXIMATION EVERYWHERE IT APPEARS

This assumes a station's share of its neighbourhood is stable across the day, which is not exactly true. A station outside an office building and one outside a bar sit in the same cell and swap roles between 09:00 and 22:00.

Every station-level number carries `approximate: true` and the derivation method. A `share_confidence` flag marks stations with fewer than 2,000 lifetime trips, where the share is too noisy to trust. Modelling stations directly would fix this and costs a full re-run of phases 2–6 at 2,459 regions.

## Availability without dock data

The user's real question is "will there be a bike", which is about *inventory*. We forecast *trips*. With no historical dock counts, printing a bike count would imply a measurement that was never made.

Instead the app reports **flow pressure** — how hard a location is being drained relative to what is arriving:

| Verdict                  | Condition                     | What the user sees                    |
|--------------------------|-------------------------------|---------------------------------------|
| Bikes leaving fast       | Net outflow > 25% of activity | "Arrive early or expect a walk"       |
| Slightly more departures | Net outflow 5–25%             | "Usually still findable"              |
| Balanced                 | Within ±5%                    | "Inflow and outflow roughly cancel"   |
| More arrivals            | Net inflow 5–25%              | "Bikes accumulating here"             |
| Docks filling up         | Net inflow > 25%              | "Returning may be the harder problem" |

## 23. Finding places without a maps API

To answer "I'm near Times Square", the app must convert a place name into coordinates. Normally this means a paid geocoding API. This project uses **its own data** instead.

Two sources already existed:

- **2,459 station names** from the registry — almost all NYC intersections ("W 21 St & 6 Ave"), which is how people describe where they are in Manhattan.
- **19,011 geocoded landmarks** from the events pipeline — parks, plazas, venues, each already resolved to a coordinate with a borough cross-check.

Matching normalises the text (expanding "avenue" to "av", stripping ordinals so "21st" matches "21"), then scores on token overlap with a substring bonus. The whole index builds in 0.19 seconds.

| Query                     | Best match                      | Source   |
|---------------------------|---------------------------------|----------|
| "Times Square"            | TIMES SQUARE, New York, NY      | landmark |
| "W 21 St and 6 Ave"       | W 21 St & 6 Ave                 | station  |
| "i am near Prospect Park" | West Drive & Prospect Park West | station  |
| "Wall Street"             | no confident match              | —        |

#### ADMITTING THE GAPS

"Wall Street" genuinely fails — Citi Bike names that area differently ("Broad St & Water St"). Rather than returning the nearest weak match and teleporting the user to Brooklyn, matches below a confidence floor are flagged, and the assistant asks the user to confirm.

A gazetteer built from station names has real holes. Saying so is better than guessing.

## 24. The assistant and its tools

The assistant has **six tools** and **three interchangeable brains**.

### The tools

| Tool                          | What it does                                                 |
|-------------------------------|--------------------------------------------------------------|
| <code>find_place</code>       | Locate a place from our own gazetteer                        |
| <code>resolve_time</code>     | Turn "now" or a date into a valid forecast interval          |
| <code>station_forecast</code> | Forecast the nearest stations with an availability verdict   |
| <code>explain_forecast</code> | Why the forecast says what it does                           |
| <code>compare_models</code>   | Run every model on the same instant, with the actual outcome |
| <code>city_overview</code>    | Every cell's forecast — what the map draws                   |
| <code>route_to_bike</code>    | Walking directions to the best dock                          |

### The three brains

- **Mistral** ( `mistral-large-2512` ) — used when `MISTRAL_API_KEY` is set in the environment.
- **Claude** — used when `ANTHROPIC_API_KEY` is set.
- **A deterministic router** — a keyword-based dispatcher that calls the same tools. No key, no network, no cost. It can't hold a free-form conversation, but it answers the questions the app exists to answer, so a fresh clone works with zero setup.

All three return an identical payload shape, so the frontend never knows which one ran. If an LLM call fails — bad key, rate limit, network blip — it degrades to the router rather than returning an error page.

#### DESIGN RULE: A TOOL NEVER NARRATES

Each tool returns numbers *plus the metadata needed to describe them honestly* — what was modelled, what was derived, what is only context. The language model does the wording; it cannot invent away the caveats, because they arrive attached to the data.

### The honesty problem this design solves

The natural thing for a chatbot to say is: *"demand is high because there's a street fair nearby."* It sounds intelligent and helpful.

**It would also be a fabrication.** The ablation tested events twice — at two different resolutions, the second with four times the data coverage — and measured their effect at  $-0.01\%$ . The deployed model does not use events at all. Attributing demand to a street fair would be inventing a cause the model never consulted.

So the tools return two clearly separated blocks:

- **drivers** — features the model genuinely consumes. Real example from the system: *"the last hour here averaged 91.0 pickups per 15 min, 27% above this cell's usual Wednesday 17:00 level (71.5)", "the same slot last week saw 151 pickups", "Wednesday 17:00 — evening rush, autumn"*.
- **context** — nearby events and weather, each carrying `influences_forecast: false` and a note explaining the ablation result.

The system prompt states these rules as non-negotiable, and they held in testing: the Mistral brain independently produced *"This is a flow forecast, not a live bike count. Numbers are derived from the cell's predicted demand split by each station's historical share."*

## Handling time honestly

The models cover January 2023 to December 2024. "Now" is outside that range. Rather than pretending, a request for the current time is mapped to the **same weekday and clock time in the matching week of 2024**, and labelled `typical_conditions`. That label survives all the way to the interface.

## 25. Walking directions with A\*

---

The app answers "take me to a bike" with real walking directions. Two graphs do two different jobs.

### Graph 1 — which dock should you walk to?

Nodes are Citi Bike stations; edges join docks within 750 m of each other, six neighbours each. Candidates are scored on walking distance **plus a penalty derived from the demand forecast**:

```
score = walking_metres + availability_penalty_metres
```

A dock that is draining fast costs up to 550 extra "virtual metres". So a dock 200 m further away with bikes actually arriving can win. **This is the point of putting a forecast behind a router** — the nearest dock is not always the right one.

### Graph 2 — how do you actually walk there?

The first implementation drew a straight line between docks, which cut across buildings. That was honest but not useful.

The real fix pulls the **walkable street network from OpenStreetMap** via the Overpass API — every road in a small bounding box, minus motorways and anything tagged `foot=no` or access-restricted. Each consecutive pair of nodes in a way becomes a bidirectional edge weighted by true ground distance.

That graph is fed to **the same A\* implementation**. Identical algorithm, real edges.

| Measure                    | Straight line (old) | Street network (now)              |
|----------------------------|---------------------|-----------------------------------|
| Times Sq → W 45 St & 8 Ave | 387 m estimated     | <b>303 m measured</b>             |
| Waypoints                  | 2                   | <b>17, following real streets</b> |
| First request in an area   | instant             | ~4 s (1.3 MB from Overpass)       |
| Cached repeat              | —                   | <b>0.11 s</b>                     |

Notice the old distance was also *wrong* — the straight-line-times-1.3 estimate had overstated it by 28%.

## Why A\* and not Dijkstra

A\* is Dijkstra plus a heuristic that estimates remaining distance to the goal, so the search expands nodes toward the destination instead of in all directions. The heuristic here is great-circle distance, which is **admissible** — it can never overestimate, because no real path is shorter than a straight line. Admissibility is what guarantees A\* still finds the optimal path.

This is the same family of algorithm Google Maps uses; production systems add contraction hierarchies, which is an index over the same shortest-path problem.

### GRACEFUL DEGRADATION

No network, an Overpass rate limit, or a point with no mapped streets nearby all fall back to the straight-line path with `follows_streets: false`. That flag reaches the interface. A straight line drawn as if it were a walking route is the kind of small lie that makes everything else look untrustworthy.

## 26. The interface

---

The layout follows ride-hailing and delivery apps: the map fills the screen, and a bottom sheet slides up with results.

### Components

- **Floating search bar** with live suggestions from the offline gazetteer.
- **Bottom sheet** that expands and collapses, holding forecast cards and the chat.
- **Selector chips** for model, spatial grid, forecast horizon, and time.
- **Station cards** showing predicted pickups and returns, a colour-coded availability pill, and a footer stating the derivation method.
- **Model comparison bars** including the actual outcome as a black bar.

### The map

Three problems were fixed here, and two shared a cause.

**Vector tiles, not raster.** The original raster basemap meant blurry text on zoom and — critically — the demand overlay had to sit *on top of everything*. With vector styles the choropleth is inserted *beneath the label layers*, so street and place names stay readable on top of the data.

**Four basemaps, no API key required:** OpenFreeMap Liberty (full detail), CARTO Positron, Voyager and Dark Matter. Setting `MAPTILER_KEY` or `STADIA_KEY` unlocks those providers, but nothing breaks without them.

**The demand grid is off by default.** Originally the hexagons covered the map at 55% opacity, hiding the buildings, shop names and street labels that make a map worth looking at. Now the choropleth is a lens you turn on. Searching a place pulses just that one cell translucently.

**Real stations on the map.** 2,391 Citi Bike docks — dots from zoom 12.5, real names from zoom 14.6, drawn above the street labels so a dock is never hidden. Click one to forecast it.

### Motion

When a route is found, the map *pulls back first* so the whole path enters frame from a wider view, then flies in to fit the route. The path draws with a soft glow underneath and an animated dashed overlay showing direction of travel.

## 27. Bugs worth remembering

---

These are included because they are the most instructive part of the project. Each was caught by a check that existed specifically to catch that class of problem.

## 1. The silent-drop bug in de-duplication

The de-duplication step used `WHERE key NOT IN (SELECT ...)`. In SQL, when the probed key is NULL this evaluates to **NULL, not TRUE** — so any row with a NULL key was filtered out with **no rule attributed to it**.

That is precisely the unattributed silent drop the cleaning phase was designed to forbid.

**How it was caught:** the traffic cleaning waterfall failed to reconcile —  $7 + 3$  did not equal 12. The arithmetic identity that every removed row must be attributed to exactly one rule is what exposed it.

It was latent in the trips pipeline too, and only escaped notice there because `ride_id` happened to have zero nulls in this particular dataset.

## 2. Daylight saving bucketing in the wrong timezone

`date_trunc('hour', started_at)` truncates in the database *session* timezone. Set to `America/New_York`, it folded both passes of the repeated autumn hour onto one label and shifted the reported counts by an hour — making the quality report contradict the (correct) pipeline. Fixed by bucketing explicitly via `AT TIME ZONE 'UTC'`.

## 3. The partitioned write that silently discarded 97% of the data

Polars' `write_parquet(partition_by=...)` names every output file `00000000.parquet` and rewrites the directory. Each batch therefore **silently replaced the previous one**. Only 35 of 1,483 cells survived.

Writes now go through DuckDB with a per-batch filename pattern, and the script asserts that the row and cell counts *on disk* match what was handed to the writer.

## 4. Evaluating on the wrong slice of validation

Covered in section 15: scoring on the first N anchors meant scoring on empty overnight intervals, reporting an MAE of 0.665 where the truth was 1.271.

## 5. Comparing a station forecast against a cell actual

The model comparison tool showed 9 predicted against 105 actual, which looked like a catastrophic model failure. It was a units bug: the prediction was *station-level and cumulative*, the actual was *cell-level and single-interval*. Two mismatches at once, understating every model roughly fourfold. All figures are now the same granularity and the same interval.

## 6. A stray word teleporting the user to Queens

Asking "why is that the case?" made the assistant answer about a station in Queens. The place-extractor stripped filler words, leaving "case", which confidently matched "**Case St & 94 St**".

Follow-up openers now require a much stronger match to override the place already under discussion — while still allowing "and what about Prospect Park?" to move correctly.

## 7. Station shares all computing to exactly zero

The station registry stores trip counts as `Decimal(scale=0)`. Dividing two of those keeps scale 0, so `302999 / 2034403` truncated to **0**. Every station share was zero and every station forecast was zero. Fixed by casting to float before any arithmetic.

## 8. Arguments parsed but never applied

Two scripts accepted a `--resolution` flag and never used it, so they would silently evaluate H3-9 while being asked for H3-8. Found by reading the code before running an expensive job.

## 28. Honest limitations

---

Everything below is a real constraint on what this project can claim.

### Data limitations

- **No dock occupancy data exists** for 2023–24. Capacity and inventory are statistical estimates from flow. Every number derived from them is flagged.
- **Traffic coverage is thin and biased.** The sensor network covers arterials, highways and bridges. Only 46 of 128 links sit within 500 m of any Citi Bike station. Manhattan is well covered; Queens barely. All 27 Staten Island links are irrelevant — no Citi Bike service.
- **The traffic feed has real multi-day outages** — 45 days with zero readings, 6.2% of the window. Carried as an explicit missing-data flag, never imputed as free-flowing.
- **24.6% of traffic readings are invalid** (status –101, filler speed 0) and are dropped by a named rule. Keeping them would drag every average toward zero.
- **Events are permits only** — no ticketed arena events at Madison Square Garden or Barclays Center, which are plausibly the events that would matter most.
- **25% of event locations could not be geocoded** from free text.
- **255 stations that really operated** in 2023–24 are missing from the official roster snapshot, which is why the station registry is derived from trip activity instead.

### Modelling limitations

- **The deep models are undertrained.** Eight epochs, all still improving. The RQ3 result is "LightGBM beats an 8-epoch TFT and ST-GNN", not a general claim about deep learning.
- **Training subsamples.** 104 million rows do not fit in 16 GB as a dense matrix, so training uses a fraction of rows. Validation and test are always scored in full.
- **No per-station model.** Station answers are disaggregated from cell forecasts.
- **MAPE is deliberately absent.** Most cells are zero at any moment, and every workaround for dividing by zero changes what is being measured.

### Application limitations

- **Nothing is live.** The models cover 2023–24. "Now" maps to the equivalent 2024 weekday and time and is labelled as typical conditions.
- **The gazetteer has real holes** — it is built from station names, so areas without stations are missing.
- **The first route in a new area takes ~4 seconds** while OpenStreetMap responds. Cached thereafter.
- **Not deployed to a public server.** It runs locally.

### What would make this materially better

1. **Live GBFS feed.** Citi Bike publishes real-time station status — bikes and docks available — free and without a key. That would turn availability from an inference into a measurement, and is the single highest-value

addition.

2. **Train the deep models to convergence** and settle RQ3 properly.
3. **Station-level modelling** to remove the disaggregation approximation and make the rebalancing question answerable.
4. **A spike-aware loss** targeting the one characteristic failure mode: under- forecasting afternoon-rush peaks in high-volume cells.

## 29. How to run everything

---

### The application

```
# from the repository root
python -m uvicorn app.server:app --port 8000
# then open http://127.0.0.1:8000
```

It runs with no configuration at all — the deterministic router answers questions, and all four basemaps are keyless. To enable a language-model brain, set a key in the environment **before** starting the server (never in the repository):

```
export MISTRAL_API_KEY="..."      # or ANTHROPIC_API_KEY
export MAPTILER_KEY="..."         # optional, extra basemaps
```

### Reproducing the research

|                                   |                                          |
|-----------------------------------|------------------------------------------|
| scripts/01_ingest.py              | → typed Parquet from the raw CSVs        |
| scripts/02_clean.py               | → cleaning waterfall + quality report    |
| scripts/03_fetch_weather.py       | → hourly weather                         |
| scripts/04_prepare_events.py      | → event permits                          |
| scripts/07_fetch_traffic.py       | → traffic via the Socrata API            |
| scripts/09_phase1_check.py        | → verifies phase 1 against the artifacts |
| scripts/10_build_panel.py         | → the dense 104M-row panel               |
| scripts/12_build_features.py      | → basic features                         |
| scripts/13_train_baseline.py      | → Seasonal Naive + LightGBM              |
| scripts/14_phase3_check.py        | → the stop-and-verify gate               |
| scripts/15_geocode_events.py      | → geocode event locations                |
| scripts/17_build_features_full.py | → all 168 features                       |
| scripts/18_ablation.py            | → the A-H family ablation                |
| scripts/19_spatial_experiment.py  | → resolution sweep + S2 matching         |
| scripts/20_export_deep_bundle.py  | → tensors for the GPU                    |
| scripts/22_train_final.py         | → tuned LightGBM                         |
| scripts/23_evaluate_test.py       | → the single test-set evaluation         |
| scripts/24_operations.py          | → capacity, inventory, rebalancing       |

# Tests

```
python -m pytest tests/ -q      # 166 passing
```

| Test file                 | Covers                                                  |
|---------------------------|---------------------------------------------------------|
| test_features.py          | 15 leakage tests re-deriving every lag independently    |
| test_advanced_features.py | 20 tests over the eight feature families                |
| test_timezone.py          | DST handling at all four transitions                    |
| test_clean_rules.py       | Every cleaning rule and waterfall reconciliation        |
| test_panel.py             | Density, target shifts, active-region selection         |
| test_spatial.py           | H3 and S2 indexers agree with reference implementations |
| test_serving.py           | The prediction interface contract                       |
| test_streets.py           | Street graph construction, offline                      |

## 30. Glossary

| Term                        | Meaning                                                                                                                                     |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MAE</b>                  | Mean Absolute Error. Average size of the mistake, in bikes per interval. Lower is better.                                                   |
| <b>RMSE</b>                 | Root Mean Squared Error. Like MAE but punishes large mistakes more.                                                                         |
| <b>WAPE</b>                 | Weighted Absolute Percentage Error. Total error divided by total demand — a relative measure that survives zeros.                           |
| <b>Hotspot F1</b>           | How well the model identifies the busiest 10% of cells at each moment. Combines precision and recall.                                       |
| <b>Skill score</b>          | $1 - (\text{model error} / \text{baseline error})$ . How much better than the obvious guess, on a scale that survives changes in cell size. |
| <b>Data leakage</b>         | Accidentally giving a model information from the future. Produces great scores and useless models.                                          |
| <b>Ablation</b>             | Adding or removing feature groups one at a time to measure what each contributes.                                                           |
| <b>H3</b>                   | Uber's hexagonal grid system for dividing the world into cells.                                                                             |
| <b>S2</b>                   | Google's equivalent, using quadrilaterals.                                                                                                  |
| <b>Panel</b>                | The dense table with one row per cell per time interval.                                                                                    |
| <b>Parquet</b>              | A compressed columnar file format. Reading two columns doesn't touch the rest of the file.                                                  |
| <b>LightGBM</b>             | Gradient-boosted decision trees — many small trees, each correcting the last.                                                               |
| <b>TFT</b>                  | Temporal Fusion Transformer. A neural network for forecasting with static, known-future and observed-past inputs.                           |
| <b>ST-GNN</b>               | Spatio-Temporal Graph Neural Network. Models cells as a graph so neighbours can inform each other.                                          |
| <b>Poisson objective</b>    | A loss function designed for count data — non-negative integers, often zero.                                                                |
| <b>A*</b>                   | A pathfinding algorithm: Dijkstra plus a heuristic that steers the search toward the goal.                                                  |
| <b>Admissible heuristic</b> | One that never overestimates remaining distance. Required for A* to find the true shortest path.                                            |
| <b>Overpass API</b>         | A query service for OpenStreetMap data.                                                                                                     |
| <b>GBFS</b>                 | General Bikeshare Feed Specification — the live standard Citi Bike publishes station status in.                                             |

## IN CLOSING

The most valuable output of this project is not the 1.14 MAE. It is the set of decisions where the honest answer was less impressive than the available one: reporting that rebalancing didn't help, that two apparent deep-learning wins were noise, that traffic and weather and events earned nothing across two separate tests, and that we cannot tell you how many bikes are at a dock because nobody published that data.

A forecasting system is only worth building if people can trust its numbers. Every caveat in this report is there so that the numbers which *are* claimed can be relied on.